

Multimedia Tycoon — Resumen técnico y guía de ejecución

Versión del documento: 1.0

Proyecto: prototipo educativo de juego móvil con Expo + React Native (JavaScript)

1. Resumen ejecutivo

Multimedia Tycoon es una aplicación móvil que simula dirigir un estudio multimedia. El jugador elige contratos de clientes ficticios y completa minijuegos asociados a roles reales de la ingeniería multimedia (diseño UI/UX, audio, desarrollo frontend). El objetivo pedagógico es relacionar roles profesionales con actividades prácticas en una interfaz colorida tipo cartoon.

La aplicación no utiliza backend: todos los datos provienen de archivos JSON locales empaquetados con la app. El estado de juego (monedas, XP, logros desbloqueados) se gestiona en memoria con React Context.

2. Qué se construyó en la aplicación

2.1 Pantallas

Pantalla	Función
Splash	Intro animada y transición al hub principal
Home	Logo/mascota, contadores de monedas y XP, acceso a juego y a teoría de roles
Selección de proyectos	Lista de clientes con diálogos humorísticos y recompensas
Minijuego	Contenedor que carga el juego según el tipo de contrato
Resultados	Estrellas, monedas y XP ganados; mención de logros
Roles	Tarjetas educativas por perfil profesional (herramientas, dificultad, curiosidades)
Ajustes	Silenciar música de fondo (preferencia global de audio)

2.2 Minijuegos implementados

- UI Fixer** — Enseña buenas prácticas de **UI/UX** (legibilidad, contraste, botones, feedback).
- Audio Match** — Asocia clips de sonido con etiquetas (**Audio Designer**); usa archivos WAV locales y `expo-av`.
- Bug Smasher** — Aplastar “bugs” con mensajes de error humorísticos (**Frontend Developer**).

2.3 Componentes reutilizables destacados

`GameButton`, `RoleCard`, `ScoreBoard`, `AnimatedMascot`, `XPBar`, `CoinCounter`, `ModalReward`, `ScreenBackground`, `LoadingOverlay`, `EmptyState`.

2.4 Sistemas transversales

- Navegación:** React Navigation (stack nativo).
- Audio:** módulo central `AudioManager` con `expo-av` (efectos en caché, música en bucle).
- Animaciones:** `react-native-animated` y `react-native-reanimated` (barra XP).
- Datos mock:** JSON en `src/data/` (proyectos, roles, minijuegos, sonidos, logros, escenarios UI).

3. Cómo está hecho (arquitectura)

3.1 Principios

- **Componentes funcionales** y **hooks** (sin clases).
- Separación entre **pantallas** (`src/screens/`), **lógica de minijuegos** (`src/games/`), **UI compartida** (`src/components/`) y **datos** (`src/data/`).
- **Estado global** del juego en `GameProvider` (`src/hooks/useGameProgress.js`).
- **Preferencias de audio** en `AudioPrefsProvider` (`src/hooks/useAudioPrefs.js`).

3.2 Punto de entrada

- Expo carga `App.js` en la raíz del proyecto, que importa `src/App.js`.
- `src/App.js` envuelve la app con `GestureHandlerRootView`, `SafeAreaProvider`, proveedores de estado y `RootNavigator`.

3.3 Flujo de navegación típico

Splash → Home → (Proyectos | Roles | Ajustes) → Minijuego → Resultados → Home (reset del stack).

3.4 Datos “tipo backend”

Los JSON se importan con `require()` en `src/data/index.js`, simulando respuestas de API sin red.

3.5 Assets de audio

Archivos WAV en `src/assets/sounds/`. El mapa clave → archivo está en `src/audio/soundAssets.js`; el comportamiento de reproducción en `src/audio/AudioManager.js`.

4. Tecnologías utilizadas

Categoría	Tecnología
Framework móvil	React Native
Toolkit	Expo (SDK ~54)
Lenguaje	JavaScript (sin TypeScript en el código fuente del juego)
Navegación	@react-navigation/native , @react-navigation/native-stack
Audio	expo-av
Animación	react-native-reanimated , react-native-animated
Gestos / raíz	react-native-gesture-handler
Áreas seguras	react-native-safe-area-context
Pantallas nativas	react-native-screens
Feedback	expo-haptics (botones)
Otros Expo	expo-splash-screen , expo-status-bar

5. Estructura de carpetas (referencia)

```
App.js
src/
App.js
assets/sounds/ # WAV de efectos y música
audio/ # AudioManager, soundAssets
components/ # UI reutilizable
```

```
screens/ # Pantallas
navigation/ # Stack y nombres de rutas
data/ # JSON mock + index.js
hooks/ # Context de juego y audio
styles/ # Tema y estilos globales
utils/ # Cálculos de estrellas / recompensas
games/ # Minijuegos
animations/ # Presets de animación
```

6. Pasos para ejecutar el proyecto

6.1 Requisitos previos

- **Node.js** (LTS recomendado) y **npm**.
- Para emulador **iOS**: macOS con Xcode.
- Para emulador **Android**: Android Studio con AVD.
- En dispositivo físico: aplicación **Expo Go** (App Store / Play Store).

6.2 Instalación de dependencias

Abrir una terminal en la carpeta del proyecto y ejecutar:

```
npm install
```

6.3 Arrancar el servidor de desarrollo

```
npx expo start
```

Se abrirá la interfaz de Expo (terminal o navegador). Desde ahí:

- Pulsar **i** para abrir en simulador iOS (si aplica).
- Pulsar **a** para abrir en emulador Android (si aplica).
- Escanear el **código QR** con Expo Go en el teléfono (misma red Wi-Fi que el ordenador).

6.4 Atajos útiles

```
npx expo start --ios
npx expo start --android
npx expo start --web
```

6.5 Lint del código

```
npx expo lint
```

7. Documentación adicional en el repositorio

En la raíz del proyecto existe un README.md con explicación educativa del propósito de cada pantalla, detalle del sistema de audio y extensión posible del proyecto.

8. Nota sobre este PDF

La fuente de este documento es `docs/RESUMEN\_MULTIMEDIA\_TYCOON.md`. Para regenerar el PDF en máquinas con Python 3:

```
pip install -r scripts/requirements-pdf.txt
python3 scripts/build_pdf_summary.py
```

El archivo de salida es `docs/RESUMEN\_MULTIMEDIA\_TYCOON.pdf`. El script usa ReportLab y, en macOS, la fuente Arial Unicode del sistema para soportar español y símbolos; en Linux suele usarse DejaVu Sans si está instalada.

Fin del documento.

Documento generado automáticamente desde docs/RESUMEN_MULTIMEDIA_TYCOON.md